

1. Define the System Goal and Threat Model

Before coding anything further, clearly define what the system protects and from whom.

Goal:

Detect and isolate cloned or compromised IoT devices using telemetry analysis, firmware verification, and immutable logging.

Threat scenarios to simulate:

- Device cloning (same firmware copied to another ESP32)
- Fake telemetry injection
- Sensor data manipulation
- Replay attacks (re-sending old telemetry)
- Firmware tampering

Write these in your project documentation.

Industry systems always start with a **threat model**.

2. Hardware Setup

Build the physical IoT environment.

Components

- 2 ESP32 boards
- 2 DHT11 sensors
- Breadboards + wires

- WiFi network
- Laptop/server

Device roles

- ESP32 #1 → Legitimate device
- ESP32 #2 → Ghost/attacker device

Both devices:

- Collect environmental data
- Send telemetry to backend

This creates the **data generation layer**.

3. Implement Real Telemetry Collection

Each ESP32 must send **real sensor telemetry**.

Telemetry fields should include:

device_id
timestamp
temperature
humidity
firmware_hash
packet_interval

The sensor data comes from:

- DHT11 temperature
- DHT11 humidity

These values will be used by the anomaly detection model.

At this stage:

- Send telemetry every 5 seconds.

4. Secure Telemetry Communication (MQTT + TLS)

Industry IoT rarely uses plain HTTP.

Instead we use:

MQTT over TLS

Architecture:

ESP32 devices



MQTT Broker



Telemetry Processing Server

Tools to use:

- Mosquitto MQTT broker
- TLS certificates
- ESP32 MQTT library

Telemetry topic example:

iot/devices/telemetry

Payload example:

```
{  
"device_id":"ESP32_01",  
"temperature":27.5,  
"humidity":60,  
"firmware_hash":"abc123",
```

```
"timestamp":"2026-03-09T10:22"
}
```

TLS ensures:

- Data cannot be intercepted
 - Devices are authenticated
-

5. Device Identity & Firmware Verification

Each device must prove its firmware integrity.

Steps:

1. Generate firmware hash during build.
2. Store expected hash in server database.
3. Device sends hash in telemetry.
4. Server compares hash.

If mismatch occurs → device marked suspicious.

This step protects against **tampered firmware**.

6. Telemetry Ingestion Layer

Create a backend service that receives device data.

Recommended stack:

- Python FastAPI or Flask
- MQTT subscriber client

- PostgreSQL / InfluxDB database

Responsibilities:

Receive telemetry

Validate schema

Store data

Forward to ML analysis

This layer is called **Telemetry Ingestion API**.

7. Data Storage Layer

Store telemetry for training and monitoring.

Recommended database types:

Time-series database

- InfluxDB
or
- TimescaleDB

Store fields:

device_id

temperature

humidity

timestamp

interval

firmware_hash

These logs become the dataset for anomaly detection.

8. Build a Machine Learning Anomaly Model

Now implement behavior analysis.

Use an **unsupervised anomaly detection model** because attacks are rare.

Good algorithms:

- Isolation Forest
- Autoencoder
- One-Class SVM

Recommended library:

scikit-learn

Features for the model:

temperature
humidity
packet_interval
rate_of_change
device uptime

Dataset sources:

- Your collected telemetry
- Kaggle IoT datasets (optional)

Example Kaggle dataset:

IoT Sensor Data for Predictive Maintenance.

Train model to learn **normal device behavior**.

Output:

anomaly_score

If score exceeds threshold → suspicious device.

9. Implement Real-Time Inference

After training, run the model live.

Pipeline:

Telemetry received



Feature extraction



ML anomaly model



Score output

Classification:

score < threshold → trusted

score ≥ threshold → suspicious

This step performs **behavioral anomaly detection**.

10. Decision Engine

Combine multiple signals.

Inputs:

- Firmware hash check
- ML anomaly score
- Packet frequency
- Device identity

Decision logic:

If firmware mismatch → quarantine

Else if anomaly_score high → suspicious
Else → trusted

Output status:

TRUSTED
SUSPICIOUS
QUARANTINED

11. Blockchain Logging Layer

Purpose: make detection logs **tamper-proof**.

Use a lightweight blockchain approach.

Options:

1. Hyperledger Fabric
2. Private Ethereum
3. Simple hash-chain logging

Store events like:

device_id
firmware_hash
anomaly_score
verdict
timestamp

Once written, logs cannot be modified.

This helps with:

- forensic analysis
 - regulatory compliance
 - audit trails
-

12. Device Response / Network Quarantine

When a device is suspicious:

Possible responses:

- Block device MAC in router
- Ignore telemetry
- Send alert
- Mark device as quarantined

In real networks this is done using:

- SDN controllers
- firewall rules

For your project:

- simply flag device and show status in dashboard.
-

13. Monitoring Dashboard

Build a visualization interface.

Use:

- React
or
- simple Flask dashboard

Display:

Device list

Temperature graphs
Anomaly score
Device status
Blockchain log

Color coding:

Green → Trusted
Yellow → Suspicious
Red → Quarantined

This becomes the **Security Operations View**.

14. Attack Simulation

Now test the system.

Ghost device can simulate:

1. Constant sensor values
2. Unrealistic temperature values
3. Faster packet transmission
4. Replay old packets

The ML model should detect these anomalies.

15. System Evaluation

Measure system performance.

Metrics:

Detection accuracy
False positive rate
Detection latency

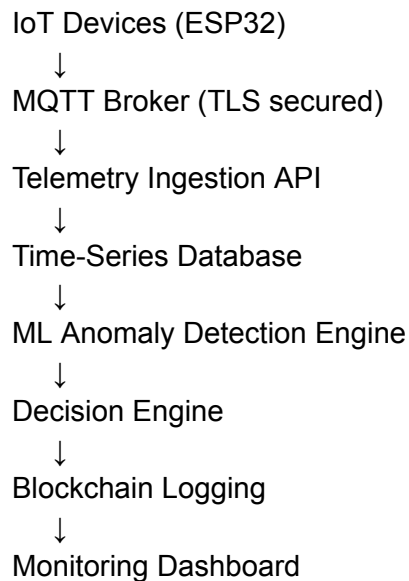
Network overhead

Compare:

- normal device
 - ghost device behavior.
-

16. Deployment Architecture

Final system architecture:



17. Documentation and Demo Preparation

Prepare:

- architecture diagram
- workflow
- dataset description

- ML model explanation
- attack scenarios
- demo script

This makes the project look **industry-grade**.

Final Outcome

The completed system will:

- Secure telemetry using MQTT + TLS
- Verify firmware integrity
- Detect anomalous device behavior using ML
- Record security events on blockchain
- Provide real-time monitoring dashboard
- Automatically flag cloned devices